CAHIER DES CHARGES

Projet Fil Rouge – SkillHub

BLOC 03

Cloud, DevOps et Architecture

Bachelor Concepteur Développeur Web Full Stack · Promotion 2025/2026

Référence : CDC-SKILLHUB-2026 · Statut : Approuvé – Diffusion restreinte

1. Présentation du projet

SkillHub est une plateforme web collaborative de mise en relation entre formateurs et apprenants. Les Blocs 01 et 02 ont couvert le développement front-end (React.js) et back-end (API REST, PostgreSQL, MongoDB). Le Bloc 03 porte sur l'industrialisation de la plateforme : hébergement cloud, conteneurisation, automatisation CI/CD et stratégie d'architecture.

Projet	SkillHub – Phase 3 : Industrialisation
Bloc	Bloc 03 – Cloud, DevOps et Architecture
Formation	Bachelor Concepteur Développeur Web Full Stack
Mode	Projet en groupe de 3 étudiants
Pré-requis	Codebase SkillHub fonctionnelle issue des Blocs 01 et 02
Stack	React.js · API REST (JWT) · PostgreSQL ou MySQL · MongoDB
Cible	Docker · CI/CD · Hébergement cloud

1.1 Périmètre

Domaine	Dans le périmètre	Hors périmètre
Audit & architecture	Rapport d'audit cloud, schéma d'architecture, plan budgétaire	Nouvelles fonctionnalités métier
Versionning	Stratégie Git, Conventional Commits, CONTRIBUTING.md	Refonte du code applicatif
Conteneurisation	Dockerfiles, docker-compose.yml, .env.example	Modification des modèles de données
CI/CD	Pipeline CI (lint, tests, build), CD (push registry)	Paiements, OAuth, SSO
Orchestration	Health checks, limites ressources, scaling documenté	Analytics avancés, refonte UX
Sécurité DevOps	Gestion des secrets, images taguées	Audit RGPD complet

2. Organisation du projet en groupe

2.1 Contexte

Le Bloc 03 est réalisé en groupe de 3 étudiants — contrairement aux Blocs 01 et 02 qui étaient individuels. Ce choix reflète la réalité professionnelle : les décisions d'architecture cloud, la mise en place d'un pipeline CI/CD et la conteneurisation d'une application sont des activités d'équipe.

Chaque étudiant arrive avec son propre dépôt SkillHub issu des Blocs précédents. Le groupe ne fusionne pas trois projets — il choisit un dépôt de référence et y contribue collectivement via des branches Git.

> **Les épreuves certifiantes EC05 et EC06 restent individuelles (4h). Un étudiant qui n'a pas contribué activement au projet groupe sera en difficulté le jour J.**

2.2 Mise en place du dépôt groupe

Le groupe sélectionne le dépôt individuel qui servira de base selon les critères suivants, par ordre de priorité :

- API JWT fonctionnelle avec PostgreSQL (ou MySQL) et MongoDB opérationnels
- Tests unitaires exécutables
- Code lisible et structuré, framework maîtrisé par le groupe

> **Le dépôt de référence doit être choisi et le dépôt groupe initialisé dans les 48h suivant le début du bloc. Passé ce délai, le formateur désigne le dépôt de référence.**

Procédure d'initialisation :

- Le propriétaire du dépôt retenu crée un nouveau dépôt groupe dédié (ex. skillhub-groupe-X), distinct de son dépôt individuel, et y pousse le code
- Il ajoute les deux autres membres comme collaborateurs avec droits d'écriture
- Chaque membre clone le dépôt groupe et configure son identité Git locale
- La branche dev est créée immédiatement à partir de main et définie comme branche de travail par défaut
- Le fichier CONTRIBUTING.md est rédigé collectivement dès le premier jour

> *En cas de désaccord sur le choix du dépôt, le formateur tranche. Signalez la situation dès le premier jour.*

2.3 Rôles

Chaque membre est responsable d'un rôle principal. La répartition doit figurer dans le CONTRIBUTING.md.

Rôle	Responsabilités principales	Livrables pilotés
Cloud Architect	Rapport d'audit cloud, comparaison des offres, schéma d'architecture, plan budgétaire	Rapport d'audit, schéma, tableau comparatif
DevOps Engineer	Dockerfiles, docker-compose.yml, pipeline CI/CD	Dockerfiles, docker-compose.yml, .github/workflows/
Tech Lead	Versionning Git, CONTRIBUTING.md, orchestration, sécurité, coordination générale	CONTRIBUTING.md, README.md, scan vulnérabilités

2.4 Traçabilité Git

Le dépôt Git est la preuve de contribution individuelle. Les règles suivantes sont évaluées :

- Chaque membre doit avoir des commits réguliers et identifiables dans l'historique
- Les messages de commit respectent la convention Conventional Commits
- Tout développement passe par une branche feature — jamais directement sur main
- Les Pull Requests doivent mentionner l'auteur et décrire le travail effectué

> **Un historique Git avec un seul auteur pour les 3 membres, ou des commits massifs en fin de projet, entraîne une pénalité sur le critère Versionning.**

3. Partie I – Audit cloud et architecture

Le groupe produit un rapport d'audit technique à destination de la direction de SkillHub. Ce rapport guidera les décisions d'hébergement et d'architecture pour la mise en production de la plateforme.

> *Compétences évaluées dans cette partie : C18 (Évaluer les besoins et intégrer les solutions cloud) · C19 (Analyser les besoins techniques) · C20 (Administrer et rationaliser les plateformes cloud).*

3.1 Contexte et données de référence

Contrainte	Valeur cible
Disponibilité	99,5 % minimum (≈ 44h de tolérance/an)
Conformité	RGPD — données hébergées en Europe
Équipe	3 développeurs, aucun ops dédié au démarrage
Budget	Limité — montée en charge progressive
Charge N1 (lancement)	500 utilisateurs actifs simultanés
Charge N2 (18 mois)	10 000 utilisateurs actifs

3.2 Contenu du rapport

Analyse des besoins (C18 – Cr18.1)

Le rapport identifie et détaille les besoins d'infrastructure de SkillHub. Niveau de détail attendu : chaque besoin est accompagné d'une justification liée au contexte de la plateforme (pas de liste générique). Les besoins couvrent :

- Besoins fonctionnels : API REST, bases de données (PostgreSQL + MongoDB), authentification JWT, stockage des ressources pédagogiques, logs activity_logs
- Besoins non fonctionnels : temps de réponse cible (ex. < 300 ms pour 95 % des requêtes), disponibilité 99,5 %, scalabilité N1 → N2, sécurité des données, conformité RGPD
- Besoins organisationnels : budget limité, équipe de 3 développeurs sans ops dédié, absence de compétences d'administration système

Comparaison des solutions cloud (C18 – Cr18.2 & Cr18.3)

Analyse comparative d'au moins deux fournisseurs parmi : AWS, GCP, Microsoft Azure, OVHcloud, Hetzner / Scaleway. Le tableau comparatif doit couvrir pour chaque option :

- Modèle de service (IaaS / PaaS / FaaS) et services utilisés
- Régions disponibles en Europe et conformité RGPD
- Coût estimé mensuel pour la phase N1 — chiffré, basé sur les tarifs publics
- Complexité d'administration pour une équipe sans ops dédié
- Points forts et risques spécifiques à SkillHub (vendor lock-in, montée en charge, support)

La recommandation finale est argumentée : elle justifie le choix retenu au regard des objectifs stratégiques de SkillHub (coût, scalabilité, simplicité opérationnelle).

Architecture applicative (C19 – Cr19.1 & Cr19.2)

Proposition d'une architecture cible justifiée parmi : monolithique, microservices, serverless, n-tiers. Le rapport doit inclure :

- Un schéma d'architecture au format C4 — au minimum les niveaux Context et Container (voir détail ci-dessous)
- La justification du choix au regard des contraintes identifiées dans l'analyse des besoins

- Une vision à long terme explicite : comment cette architecture accompagne la croissance de SkillHub sur 3 à 5 ans, facilite la maintenance et intègre de nouvelles technologies

Modèle C4 — niveaux attendus :

Niveau	Nom	Ce qu'il représente
C1 — obligatoire	Context Diagram	SkillHub vu de l'extérieur : les utilisateurs (apprenant, formateur), les systèmes externes (navigateur, email, stockage) et leurs interactions avec la plateforme
C2 — obligatoire	Container Diagram	Les conteneurs techniques de SkillHub : React SPA, API REST, PostgreSQL, MongoDB, et leurs interactions (protocoles, ports, données échangées)
C3 — optionnel	Component Diagram	Les composants internes d'un conteneur spécifique, ex. les modules de l'API REST (Auth, Formations, Modules, Enrollments, Logs)

Outils acceptés pour la réalisation des diagrammes C4 : draw.io (structurizr plugin) · Mermaid (C4 DSL) · Structurizr Lite · PlantUML C4-PlantUML · ou tout outil équivalent permettant de produire un diagramme clair et annoté.

> *Les deux niveaux C1 et C2 sont obligatoires. Le niveau C3 est optionnel. Chaque diagramme doit inclure une légende et des annotations expliquant les technologies et protocoles utilisés.*

Plan d'administration et de sécurité (C20 – Cr20.1, Cr20.2, Cr20.3)

Trois axes couverts, chacun avec des éléments chiffrés ou des références concrètes :

- Optimisation des ressources : stratégie de scaling (horizontal / vertical), outils de monitoring envisagés (CloudWatch, Grafana, Datadog…), métriques clés surveillées, démonstration de l'équilibre coûts/performances
- Plan budgétaire : estimation mensuelle phase N1 (500 utilisateurs) et projection phase N2 (10 000 utilisateurs à 18 mois), leviers d'optimisation identifiés — le plan doit être réaliste au regard du budget de démarrage limité
- Sécurité cloud : IAM et principe de moindre privilège, chiffrement TLS en transit et au repos, gestion des secrets (Vault, Secrets Manager ou équivalent), plan de sauvegarde avec RTO et RPO définis, conformité RGPD

3.3 Structure du rapport

Page de garde · Résumé exécutif (½ page) · 1. Analyse des besoins · 2. Comparaison des solutions cloud · 3. Architecture recommandée (avec schéma) · 4. Plan d'administration et de sécurité · Annexes (optionnel)

> *Le schéma d'architecture est obligatoire. Un rapport sans schéma ne peut obtenir la note maximale sur C19.*

4. Partie II – Versionning, conteneurisation et CI/CD

> *Compétences évaluées dans cette partie : C21 (Gestion et suivi des révisions) · C22 (Fusion et intégration continue) · C23 (Environnements virtualisés) · C24 (Orchestration des conteneurs) · C25 (Automatisation du déploiement cloud).*

4.1 Versionning Git

Stratégie de branches obligatoire :

Branche	Rôle
main	Production — code stable uniquement, aucun commit direct
dev	Intégration — accumule les fonctionnalités validées
feature/<nom>	Une branche par fonctionnalité ou tâche
hotfix/<nom>	Correctifs urgents — merge vers main et dev (recommandé)

Conventions de commit (Conventional Commits) :

- feat: — nouvelle fonctionnalité
- fix: — correction de bug
- docker: — ajout ou modification des fichiers de conteneurisation
- ci: — modification du pipeline CI/CD
- docs: — documentation
- chore: — maintenance

Exemples de messages de commit attendus dans le contexte SkillHub :

```
feat(api): add JWT authentication middleware
docker: add multi-stage Dockerfile for React front-end
ci: configure GitHub Actions pipeline with lint and test stages
fix: resolve port conflict in docker-compose.yml
docs: update README with docker compose up instructions
```

Fichiers requis : .gitignore adapté à la stack · .gitattributes (LF pour Docker) · CONTRIBUTING.md documentant la stratégie de branches, le format des commits et la procédure de Pull Request.

4.2 Conteneurisation Docker

Dockerfiles

Un Dockerfile par composant (API back-end, front-end React). Exigences :

- Image de base officielle (-alpine ou -slim recommandé)
- Build multi-stage : séparation builder / runtime
- Aucune credential en dur — variables externalisées
- Fichier .dockerignore à la racine de chaque composant

Docker Compose

Le fichier docker-compose.yml orchestre la stack complète. Services requis : API + PostgreSQL + MongoDB (+ nginx reverse proxy en option). Exigences :

- Réseau interne Docker isolé
- Health checks sur les services db, mongodb et api

- Dépendances de démarrage : depends_on avec condition service_healthy
- Volumes nommés pour la persistance PostgreSQL et MongoDB
- Variables sensibles externalisées via .env — un fichier .env.example versionné documente les variables requises

> **La commande docker compose up --build doit démarrer la stack sans erreur depuis une machine vierge.**

4.3 Pipeline CI

Outil au choix : GitHub Actions · GitLab CI/CD · Jenkins. Déclenchement automatique sur push vers dev et main, et sur toute Pull Request ouverte vers ces branches.

Étapes minimales dans l'ordre :

Étape	Description
1 – Install	Installation des dépendances
2 – Lint	Analyse statique du code (ESLint, Flake8, PHP_CodeSniffer…)
3 – Test	Exécution des tests unitaires avec rapport de résultats
4 – Build image	Construction de l'image Docker taguée avec le git SHA

Sécurité obligatoire : aucune credential en clair dans le fichier de configuration — toutes les variables sensibles stockées dans les secrets du pipeline (GitHub Actions Secrets ou GitLab CI Variables).

4.4 Pipeline CD

Le pipeline CD se déclenche automatiquement après merge sur main, une fois toutes les étapes CI passées. Exigence minimale : build et push automatique de l'image Docker taguée avec le git SHA vers une registry (Docker Hub ou GitHub Container Registry).

```
docker build -t skillhub-api:${{ github.sha }} ./backend
docker push skillhub-api:${{ github.sha }}
```

Les credentials de la registry sont stockés dans les secrets du pipeline — jamais en clair dans les fichiers de configuration.

4.5 Orchestration

Le fichier docker-compose.yml doit assurer une orchestration fonctionnelle de la stack. Exigences minimales :

- Health checks définis sur les services api, db et mongodb
- Limites de ressources configurées (mem_limit, cpus) sur chaque service
- Politique de redémarrage : restart: unless-stopped
- Dépendances de démarrage : depends_on avec condition service_healthy

4.6 Sécurité

- Aucune credential en dur dans les Dockerfiles ou docker-compose.yml
- Images construites depuis des sources officielles — tags précis, pas latest
- Fichier .env jamais commité — .env.example versionné à la place
- Variables sensibles du pipeline stockées dans les secrets GitHub Actions ou GitLab CI

5. Livrables attendus

Livrable	Contenu attendu	Format	C.
Rapport d'audit	Page de garde · Résumé exécutif (½ p.) · Analyse des besoins (fonctionnels, non fonctionnels, organisationnels) · Tableau comparatif ≥ 2 fournisseurs cloud · Architecture recommandée + justification · Plan budgétaire N1/N2 · Plan sécurité (IAM, chiffrement, secrets, RTO/RPO, RGPD)	PDF ou DOCX	C18 C19 C20
Diagrammes C4	C1 Context Diagram : utilisateurs, systèmes externes, interactions · C2 Container Diagram : React SPA, API REST, PostgreSQL/MySQL, MongoDB, protocoles et ports entre services · Légende et annotations obligatoires	Intégré au rapport	C19
Dépôt Git groupe	Historique de commits avec 3 auteurs distincts · Branches main, dev, feature/* · Messages au format Conventional Commits · Pull Requests documentées	URL GitHub / GitLab	C21 C22
CONTRIBUTING.md	Répartition des rôles · Stratégie de branches · Format des commits · Procédure de Pull Request · Procédure de résolution de conflits	Markdown	C21
Dockerfile API	Image de base officielle (-alpine ou -slim) · Build multi-stage (builder + runtime) · Aucune credential en dur · Variables via ARG/ENV · Port exposé	Dans le dépôt /backend	C23
Dockerfile front-end	Stage builder : node + npm run build · Stage runtime : nginx:alpine servant le dossier dist/ · Configuration nginx pour React Router	Dans le dépôt /frontend	C23
docker-compose.yml	Services : api + db (PostgreSQL ou MySQL) + mongodb · Réseau interne isolé · Health checks sur chaque service · depends_on avec condition service_healthy · Volumes nommés pour persistance · Limites mem_limit et cpus · restart: unless-stopped	À la racine du dépôt	C23 C24
.env.example	Toutes les variables d'environnement requises listées avec une description · Aucune valeur sensible réelle · Doit couvrir : DATABASE_URL, POSTGRES_USER, POSTGRES_PASSWORD, MONGO_URI, JWT_SECRET, NODE_ENV (ou équivalents selon le framework)	À la racine du dépôt	C25
Pipeline CI	Déclenchement sur push vers main et dev + PR · Étapes dans l'ordre : install → lint → test → build image Docker taguée avec git SHA · Secrets stockés dans GitHub Actions Secrets ou GitLab CI Variables · Aucune credential en clair	.github/workflows/ ou .gitlab-ci.yml	C22 C25

Pipeline CD	Déclenchement automatique après merge sur main · Push de l'image taguée vers la registry (Docker Hub ou GitHub Container Registry) · Preuve : capture d'écran de l'image dans la registry avec tag git SHA visible	Intégré au pipeline CI	C25
README.md	Description du projet et stack · Prérequis (Docker, Git, framework) · Commande de lancement : docker compose up --build · Commande de tests : npm test / pytest / phpunit · Structure du dépôt · Variables d'environnement (.env.example) · Stratégie de branches	À la racine du dépôt	C21

5.1 Structure du dépôt

```
skillhub-groupe-X/
├── frontend/
│   ├── Dockerfile
│   ├── src/
│   └── ...
├── backend/
│   ├── Dockerfile
│   ├── src/
│   └── ...
├── .github/
│   └── workflows/
│       └── ci.yml          # ou .gitlab-ci.yml à la racine
├── docker-compose.yml
├── .env.example
├── .gitignore
├── .dockerignore
├── CONTRIBUTING.md
└── README.md
```

> *La base de données principale peut être PostgreSQL ou MySQL selon le choix de l'équipe — les deux sont acceptés. Le service db dans le docker-compose.yml doit refléter ce choix (image postgres:15-alpine ou mysql:8).*

6. Barème d'évaluation

Critère	Indicateurs
Rapport d'audit (analyse, comparaison, justification)	Besoins identifiés avec précision, ≥ 2 offres comparées et chiffrées, recommandation argumentée, plan budgétaire N1/N2
Architecture (schéma obligatoire + vision long terme)	Schéma lisible et annoté, choix justifié au regard des contraintes, projection 3-5 ans présente
Conteneurisation (Dockerfiles + docker-compose.yml)	docker compose up --build sans erreur depuis une machine vierge, bonnes pratiques respectées, .env externalisé
Pipeline CI (lint, tests, build image)	Pipeline se déclenche automatiquement, étapes distinctes, image Docker taguée avec git SHA
Pipeline CD (push image registry)	Image poussée automatiquement vers la registry après merge sur main, credentials dans les secrets
Versionning & CONTRIBUTING.md	3 auteurs distincts dans l'historique, Conventional Commits, branches respectées, Pull Requests documentées
Orchestration (health checks + limites ressources)	Health checks fonctionnels, limites CPU/mémoire définies, politique de redémarrage configurée
Sécurité	Aucune credential dans le dépôt ou le pipeline, images taguées précisément, .env non commité

– Fin du document – CDC-SKILLHUB-2026 – Bloc 03 –